ITA0608 - Machine Learning
List of Lab Exercises

15.Implement Iris Flower Classification using Naive Bayes classifier.

AIM

To implement Iris Flower Classification using the Naive Bayes classifier and classify iris flowers into Setosa, Versicolor, and Virginica based on their sepal and petal measurements.

ALGORITHM

1. Import the required Python libraries.
2. Load the Iris dataset from sklearn.datasets.
3. Separate the input features and target classes.
4. Split the dataset into training and testing data.
5. Create a Gaussian Naive Bayes classifier.
6. Train the classifier using the training dataset.
7. Predict the flower species for the test dataset.
8. Calculate the classification accuracy.
9. Predict the class of a new iris flower.

PROGRAM

DATA SET 1:

```
from sklearn.naive_bayes import GaussianNB

from sklearn.model_selection import train_test_split

from sklearn.metrics import confusion_matrix, accuracy_score

X = [

   [150,7.5,90],

   [160,7.8,88],

   [145,7.3,91],

   [120,5.5,70],
```

```
    [125,5.8,72],
    [130,6.0,74],
    [155,7.6,89],
    [118,5.4,69],
    [148,7.4,92],
    [122,5.7,71]
]
y = ["Apple","Apple","Apple","Orange","Orange",
     "Orange","Apple","Orange","Apple","Orange"]
X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=0.3,random_state=42
)
model = GaussianNB()
model.fit(X_train,y_train)
y_pred = model.predict(X_test)
print("Confusion Matrix:")
print(confusion_matrix(y_test,y_pred))
print("Accuracy:", accuracy_score(y_test,y_pred))
new_sample = [[152,7.5,90]]
print("Predicted Fruit:", model.predict(new_sample)[0])
```

OUTPUT:

```python
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score
X = [
    [150,7.5,90],
    [160,7.8,88],
    [145,7.3,91],
    [120,5.5,70],
    [125,5.8,72],
    [130,6.0,74],
    [155,7.6,89],
    [118,5.4,69],
    [148,7.4,92],
    [122,5.7,71]
]
y = ["Apple","Apple","Apple","Orange","Orange",
     "Orange","Apple","Orange","Apple","Orange"]
X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=0.3,random_state=42
)
model = GaussianNB()
model.fit(X_train,y_train)
y_pred = model.predict(X_test)
print("Confusion Matrix:")
print(confusion_matrix(y_test,y_pred))
print("Accuracy:", accuracy_score(y_test,y_pred))
new_sample = [[152,7.5,90]]
print("Predicted Fruit:", model.predict(new_sample)[0])
```

```
Confusion Matrix:
[[2 0]
 [0 1]]
Accuracy: 1.0
Predicted Fruit: Apple
```

DATA SET 2:

```
from sklearn.naive_bayes import GaussianNB

from sklearn.model_selection import train_test_split

from sklearn.metrics import confusion_matrix, accuracy_score

X = [

   [9.2,90,88],

   [8.8,85,82],

   [8.5,80,79],
```

```
    [7.2,72,70],
    [6.8,68,65],
    [9.0,92,90],
    [7.5,75,74],
    [8.6,84,81],
    [6.5,60,58],
    [8.9,88,86]
]
y = ["Placed","Placed","Placed","Not Placed","Not Placed",
     "Placed","Not Placed","Placed","Not Placed","Placed"]
X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=0.3,random_state=42
)
model = GaussianNB()
model.fit(X_train,y_train)
y_pred = model.predict(X_test)
print("Confusion Matrix:")
print(confusion_matrix(y_test,y_pred))
print("Accuracy:", accuracy_score(y_test,y_pred))
new_sample = [[8.7,86,84]]
print("Predicted Placement:", model.predict(new_sample)[0])
```

OUTPUT:

```python
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score
X = [
    [9.2,90,88],
    [8.8,85,82],
    [8.5,80,79],
    [7.2,72,70],
    [6.8,68,65],
    [9.0,92,90],
    [7.5,75,74],
    [8.6,84,81],
    [6.5,60,58],
    [8.9,88,86]
]
y = ["Placed","Placed","Placed","Not Placed","Not Placed",
     "Placed","Not Placed","Placed","Not Placed","Placed"]
X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=0.3,random_state=42
)
model = GaussianNB()
model.fit(X_train,y_train)
y_pred = model.predict(X_test)
print("Confusion Matrix:")
print(confusion_matrix(y_test,y_pred))
print("Accuracy:", accuracy_score(y_test,y_pred))
new_sample = [[8.7,86,84]]
print("Predicted Placement:", model.predict(new_sample)[0])
```

```
Confusion Matrix:
[[1 0]
 [0 2]]
Accuracy: 1.0
Predicted Placement: Placed
```

DATA SET 3:

```
from sklearn.naive_bayes import GaussianNB

from sklearn.model_selection import train_test_split

from sklearn.metrics import confusion_matrix, accuracy_score

X = [

    [39.0,110,94],

    [38.5,108,95],

    [37.0,78,99],

    [39.2,115,93],
```

```
    [36.8,75,98],
    [38.8,112,94],
    [37.2,80,98],
    [39.1,114,92],
    [36.7,74,99],
    [38.9,111,93]
]
y = ["Positive","Positive","Negative","Positive","Negative",
     "Positive","Negative","Positive","Negative","Positive"]
X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=0.3,random_state=42
)
model = GaussianNB()
model.fit(X_train,y_train)
y_pred = model.predict(X_test)
print("Confusion Matrix:")
print(confusion_matrix(y_test,y_pred))
print("Accuracy:", accuracy_score(y_test,y_pred))
new_sample = [[38.7,109,94]]
print("Predicted Disease:", model.predict(new_sample)[0])
```

OUTPUT:

```
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score
X = [
    [39.0,110,94],
    [38.5,108,95],
    [37.0,78,99],
    [39.2,115,93],
    [36.8,75,98],
    [38.8,112,94],
    [37.2,80,98],
    [39.1,114,92],
    [36.7,74,99],
    [38.9,111,93]
]
y = ["Positive","Positive","Negative","Positive","Negative",
     "Positive","Negative","Positive","Negative","Positive"]
X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=0.3,random_state=42
)
model = GaussianNB()
model.fit(X_train,y_train)
y_pred = model.predict(X_test)
print("Confusion Matrix:")
print(confusion_matrix(y_test,y_pred))
print("Accuracy:", accuracy_score(y_test,y_pred))
new_sample = [[38.7,109,94]]
print("Predicted Disease:", model.predict(new_sample)[0])
```

```
Confusion Matrix:
[[1 0]
 [0 2]]
Accuracy: 1.0
Predicted Disease: Positive
```

DATA SET 4:

```
from sklearn.naive_bayes import GaussianNB

from sklearn.model_selection import train_test_split

from sklearn.metrics import confusion_matrix, accuracy_score

X = [

   [10,95,50],

   [8,90,45],

   [7,88,40],
```

```
    [3,65,20],
    [2,60,18],
    [9,93,48],
    [4,70,25],
    [8,89,42],
    [2,58,15],
    [6,85,38]
]
y = ["Promoted","Promoted","Promoted","Not Promoted",
     "Not Promoted","Promoted","Not Promoted",
     "Promoted","Not Promoted","Promoted"]
X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=0.3,random_state=42
)
model = GaussianNB()
model.fit(X_train,y_train)
y_pred = model.predict(X_test)
print("Confusion Matrix:")
print(confusion_matrix(y_test,y_pred))
print("Accuracy:", accuracy_score(y_test,y_pred))
new_sample = [[7,90,44]]
print("Predicted Promotion:", model.predict(new_sample)[0])
```

OUTPUT:

```
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score
X = [
    [10,95,50],
    [8,90,45],
    [7,88,40],
    [3,65,20],
    [2,60,18],
    [9,93,48],
    [4,70,25],
    [8,89,42],
    [2,58,15],
    [6,85,38]
]
y = ["Promoted","Promoted","Promoted","Not Promoted",
     "Not Promoted","Promoted","Not Promoted",
     "Promoted","Not Promoted","Promoted"]
X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=0.3,random_state=42
)
model = GaussianNB()
model.fit(X_train,y_train)
y_pred = model.predict(X_test)
print("Confusion Matrix:")
print(confusion_matrix(y_test,y_pred))
print("Accuracy:", accuracy_score(y_test,y_pred))
new_sample = [[7,90,44]]
print("Predicted Promotion:", model.predict(new_sample)[0])
```

```
Confusion Matrix:
[[1 0]
 [0 2]]
Accuracy: 1.0
Predicted Promotion: Promoted
```

DATA SET 5:

```
from sklearn.naive_bayes import GaussianNB

from sklearn.model_selection import train_test_split

from sklearn.metrics import confusion_matrix, accuracy_score

X = [

   [150,7.5,90],

   [160,7.8,88],

   [145,7.3,91],
```

```
    [120,5.5,70],
    [125,5.8,72],
    [130,6.0,74],
    [155,7.6,89],
    [118,5.4,69],
    [148,7.4,92],
    [122,5.7,71]
]
y = ["Apple","Apple","Apple","Orange","Orange",
     "Orange","Apple","Orange","Apple","Orange"]
X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=0.3,random_state=42
)
model = GaussianNB()
model.fit(X_train,y_train)
y_pred = model.predict(X_test)
print("Confusion Matrix:")
print(confusion_matrix(y_test,y_pred))
print("Accuracy:", accuracy_score(y_test,y_pred))
```

OUTPUT:

```python
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score
X = [
    [150,7.5,90],
    [160,7.8,88],
    [145,7.3,91],
    [120,5.5,70],
    [125,5.8,72],
    [130,6.0,74],
    [155,7.6,89],
    [118,5.4,69],
    [148,7.4,92],
    [122,5.7,71]
]
y = ["Apple","Apple","Apple","Orange","Orange",
     "Orange","Apple","Orange","Apple","Orange"]
X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=0.3,random_state=42
)
model = GaussianNB()
model.fit(X_train,y_train)
y_pred = model.predict(X_test)
print("Confusion Matrix:")
print(confusion_matrix(y_test,y_pred))
print("Accuracy:", accuracy_score(y_test,y_pred))
```

```
Confusion Matrix:
[[2 0]
 [0 1]]
Accuracy: 1.0
```

RESULT

The Iris Flower Classification was successfully implemented using the Gaussian Naive Bayes classifier. The model classifies iris flowers into Setosa, Versicolor, and Virginica based on sepal and petal measurements. The model generally achieves an accuracy of around 95–100% on the test data, depending on the train-test split.